



Security Assessment



Item 2 Internal Report

June 2026

Prepared for ether.fi



Disclaimer

This document is intended for **internal** use only.

Initial Commit Hash	Final Commit Hash
db245879	-

Informational Issues

I-01. `_roleRegistry` could be set to the 0 address in initializer

Description: Inside the `initialize()` function of `SCRRecoveryModule.sol` the `_roleRegistry` parameter is not validated against `address(0)` and allows the immutable variable to be set to it.

Recommendation: Add a validation check in `initialize()` to ensure that `roleRegistry` cannot be set to an invalid value.

Customer's response: Acknowledged

Fix Review: Acknowledged

I-02. Hook and SCR module can be initialized with a role registry different from the `DataProvider` registry

Description: Both `EtherFiHook.initialize` and `SCRRecoveryModule.initialize` accept an arbitrary `_roleRegistry` and pass it to `__UpgradeableProxy_init`. However, the new security-sensitive functions do not use the inherited registry for authorization:

- `EtherFiHook.setScrRecoveryModule` checks `dataProvider.roleRegistry().owner()`.
- `SCRRecoveryModule.setCollectionWallet` and `collect` check roles through `dataProvider.roleRegistry()`.

There is no validation that the initializer-provided `_roleRegistry` equals `address(dataProvider.roleRegistry())`. This can create split-brain authority if the proxy is initialized with the wrong registry: inherited proxy-level controls such as upgrades or pause

mechanisms may be governed by one registry, while operational permissions are governed by the DataProvider registry.

Recommendation: In both initializers, require `_roleRegistry == address(dataProvider.roleRegistry())`. Alternatively, remove the `_roleRegistry` initializer parameter and always initialize from `dataProvider.roleRegistry()`. Use the same registry consistently for inherited proxy administration and contract-level role checks.

Customer's response: Acknowledged

Fix Review: Acknowledged

I-03. `collect` function can suffer from Out-Of-Gas (OOG) DoS due to unbounded external calls in loop

Description: In the `SCRRecoveryModule`, the `collect` function iterates over an unbounded array of safes. For each safe, the `_collect` internal function is invoked, which executes the following external calls:

- `dataProvider.isEtherFiSafe(safe)`: Queries the data provider.
- The data provider inherently makes another external call to `etherFiSafeFactory.isEtherFiSafe(safe)`.
- `scr.balanceOf(safe)`: Queries the Scroll token balance.
- `IEtherFiSafe(safe).execTransactionFromModule(...)`: Calls the safe to execute the transfer.

Because these multiple nested external calls are executed for every safe in the array, the transaction consumes higher amount of gas per iteration. If the backend wallet submits a large batch of safes, the transaction is highly susceptible to hitting the block gas limit, resulting in an Out-Of-Gas (OOG) revert. While the backend can mitigate this by chunking the array off-chain, an unexpectedly high gas consumption per iteration reduces the operational efficiency of the batching mechanism.



Recommendation: To mitigate this, enforce a maximum batch size limit (e.g., maxSafesPerBatch) either on-chain or in the BE to ensure that the transaction never exceeds the block gas limit, guaranteeing operational reliability.

Customer's response: Acknowledged

Fix Review: Acknowledged

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.